CSDM Specification Format

This document defines the YAML specification format for **Common Service Data Model (CSDM)** objects, Service Mapping population methods, runtime tags, and dependency relationships. It applies to any deployment that registers Business Applications, Business Services, and Application Services in a CMDB.

Specifications are written primarily for the **CSDM Modeler** — the person who understands the application architecture and enterprise role of each service. Automation (**deploy processor**) materializes those declarations into the CMDB.

Purpose

Operations and platform teams need a **data-driven, version-controlled** way to declare CSDM hierarchy, choose how Service Mapping populates application service maps, declare dependency relationships, and apply consistent runtime labels on Kubernetes and Docker workloads.

Authors maintain human-readable `csdm.yaml` files per application stack. The **deploy processor** creates or updates CMDB records, links relationships, registers vertical entry points when required, and triggers vertical discovery only where the specification explicitly requests it.

Audiences: **CSDM Modelers**, deploy processor maintainers, discovery operators, platform engineers applying runtime labels, and Service Mapping operators configuring tag-based rules on the instance.

Definitions

Document and processing terms

- **CSDM specification** (`csdm.yaml`): YAML declaring `business_applications`, `business_services`, and `application_services`, optional `tag_defaults`, and related attributes for one application or stack.
- **Deploy processor**: Generic automation (Ansible tasks under `csdm/tasks/`) that reads registered specification files and updates the CMDB. The deploy processor **must not** embed names, hosts, or topology from any particular solution.
- **Deploy registry**: In-memory map of object names to CMDB `sys_id` values built during a deploy run, used to resolve cross-file `depends_on` targets.
- **Deferred dependency**: A `depends_on` link whose target CI is not yet in the CMDB; the deploy processor skips the link without failing and reports it for a later run.
- **Runtime tag manifest**: Optional processor output listing merged labels per application service instance for orchestration playbooks.

CSDM object terms

- **Business Application (BA)**: Top-level CSDM object (`cmdb_ci_business_app`).
- **Business Service (BS)**: Logical service under a BA (`cmdb_ci_service`).
- **Application Service**: Deployable or mappable software unit (`cmdb_ci_service_discovered`).
- **Identifier**: Stable machine-readable key (`[a-z0-9-]+`, max 63 characters). Same concept as a web **slug**.

- **Platform**: Runtime environment — `kubernetes`, `docker`, `host`, or `saas`.

Service Mapping terms

- **Service Mapping (SM)**: ServiceNow capability that builds application service maps from CMDB CIs and relationships.
- **Tag-based Service Mapping**: SM population that groups CMDB CIs by label/tag rules (KVA labels, Docker Compose labels, `cmdb_key_value`). Does **not** use vertical discovery or `sa_m2m` entry-point registration for map construction.
- **Vertical discovery** (top-down): Classic Service Mapping starting from a registered **entry point**, walking host TCP/process/container relationships via MID Server. In specifications, `discover: true` triggers vertical discovery **only** when `service_mapping: vertical`.
- **Horizontal Discovery**: Infrastructure discovery (Linux servers, TCP, processes, Docker Pattern, K8s resources) that enriches the CMDB. CSDM deploy **does not** replace horizontal Discovery.
- **Entry point**: A `cmdb_ci_endpoint` CI linked **Depends on::Used by** from an application service and registered in `sa_m2m_service_entry_point` for **vertical** Service Mapping only.
- **Docker Pattern**: ServiceNow horizontal discovery probe that discovers Docker containers and creates the process/container relationship graph vertical Service Mapping expects. Distinct from custom container inventory sync that only upserts `cmdb_ci_docker_container` rows.

Relationship terms

- `depends_on`: Consumer → provider **Depends on::Used by** relationships declared on an application service.
- **Contains::Contained by**: BA → BS → application service hierarchy.
- **Runs on::Runs**: Endpoint CI → Linux server host linkage (vertical entry points).

Dependency tiers

Authors **should** document `depends_on` entries using tier comments (not stored as CMDB fields).

Tier	Name	Typical targets
1	Data path	Other application services
2	Host / runtime	`linux_server` CIs
3	Cross–business application	Application services in another BA
4	Infrastructure enrichment	`nfs_server`, storage, integrations

Identifier and naming

- CSDM `name` values **must not** embed **location** or **cluster** tokens (`brooks-lab`, `(Lab3)` as site).
- **Location must** use the top-level `location` attribute (CMDB `cmn_location`) and matching runtime labels.
- With `expand`, display `name` **may** include host instance suffix (`Spark Worker (Lab1)`); `identifier` **must** include the host token (`spark-worker-lab1`).

Location: CMDB column vs runtime label

Mechanism	Where it lives	Purpose
`location` (YAML top-level)	CMDB reference on BA/BS/application service CIs	Authoritative CSDM/APM placement; reporting and filters
`servicenow.io/location` (runtime label)	Kubernetes labels / Docker Compose labels → `cmdb_key_value` via discovery	Tag-based Service Mapping rules; correlates discovered workload CIs that may not inherit CMDB columns directly

Authors **must** set both to the same location name (`cmn_location.name`, for example `brooks-lab`). The CMDB column is the system-of-record for CSDM objects the deploy processor creates. Labels propagate placement to dynamically discovered CIs (pods, containers) so Service Mapping can group them without manual CMDB edits.

Roles and Responsibilities

CSDM Modeler

The **CSDM Modeler** understands CSDM, the application architecture, and each component's role in the enterprise. The CSDM Modeler **must** author and maintain `csdm.yaml` files: business hierarchy, application services, platform classification, Service Mapping method (`tags` vs `vertical` vs `manual`), `depends_on` tiers, identifiers, ownership, and runtime tag intent. The CSDM Modeler **must** register new specification file paths with the deploy processor maintainer. The CSDM Modeler **should** coordinate with platform engineers so Compose/manifest labels match the specification before expecting populated Service Maps.

Deploy processor

The **deploy processor** is the generic automation (`csdm/tasks/`, invoked by `csdm/deploy.yml`) that materializes CSDM specifications into the CMDB. It **must** validate each application service, resolve users and locations, upsert CIs, create hierarchy and `depends_on` relationships, register vertical entry points when specified, trigger vertical discovery asynchronously, and emit runtime tag manifests when configured. It **must not** embed application-specific topology.

Deploy processor maintainer

The **deploy processor maintainer** owns and extends that automation. The maintainer **must** keep the processor solution-independent and maintain the specification file registry in `csdm/common/vars.yml`.

Discovery operator

The **discovery operator** runs horizontal Discovery (SSH Linux, Docker Pattern, KVA) so infrastructure and workload CIs exist in the CMDB **before** Service Mapping is expected to succeed. The discovery operator **must not** rely on CSDM deploy alone for TCP/process/container enrichment required by vertical discovery.

Service Mapping operator

The **Service Mapping operator** configures tag-based Service Mapping rules on the ServiceNow instance (typically keyed on **servicenow.io/application-service-identifier**). The operator **must** configure rules before

tag-based application services can reach **operational** map status. The operator **should** monitor vertical discovery jobs triggered by the deploy processor and investigate services stuck in **Requirements**.

Statements

1. Service Mapping — platform guidance (ServiceNow best practice)

ServiceNow's current best practice for **Kubernetes** is **tag-based** Service Mapping via KVA labels — **not** vertical discovery. For **Docker Compose** stacks, ServiceNow **recommends tag-based** mapping; vertical discovery **may** be used but is **not required** and often fails without full Docker Pattern enrichment. For **host** workloads, vertical discovery **may** succeed when horizontal SSH discovery has enriched TCP/process data. **SaaS** services **must** use manual mapping.

> **Service Mapping population by platform**

Platform	Recommended SM method	Vertical (`discover: true`)	Vertical worth the effort?
Kubernetes	Tag-based (KVA labels)	**Must not**	**No** — ServiceNow documents KVA + tag-based SM for K8s
Docker Compose	Tag-based	**May** (optional)	**Rarely** — only when Docker Pattern enrichment is complete and ports are stable; otherwise use tags
Host (process on Linux server)	Tag-based or vertical	**May** (optional)	**Sometimes** — vertical works when entry port has `cmdb_tcp` + process linkage from SSH discovery
SaaS	Manual	**Must not**	**No**

1. Authors **must** choose `service_mapping` per platform using the table above as normative guidance.
2. Authors **must not** set `discover: true` on `platform: kubernetes` or `platform: saas`.
3. When tag-based mapping is used, authors **must** apply runtime labels on workloads **and** configure matching ServiceNow tag-based SM rules — CMDB hierarchy alone does **not** populate maps.
4. When vertical mapping is used, authors **must** satisfy **Vertical discovery prerequisites** (Statement 1.3) before expecting **Discovered** status.

1.1 Without vertical discovery — rich CSDM model

When using **tag-based** or **manual** mapping, authors **must** still declare:

- Full BA → BS → application service hierarchy with mandatory ownership attributes
- `depends_on` tiers for failure propagation and Service Map context
- `identifier`, `environment`, `location`, `service_tier` on every application service
- Runtime **servicenow.io/*** labels on workloads (and platform-specific keys below)
- Horizontal discovery / KVA / Docker inventory so backing CIs exist in the CMDB

Tag-based Service Mapping adds **Contains** relationships from application services to discovered workload CIs when instance rules match labels. `depends_on` supplies cross-service and infrastructure edges the map does not infer automatically.

1.2 Required and recommended runtime tags by platform

> **All platforms — servicenow.io keys (on workloads when `service_mapping: tags`)**

Label key	Requirement	Source in csdm.yaml
`servicenow.io/application-identifier`	**Must**	BA `identifier`
`servicenow.io/business-service-identifier`	**Must**	BS `identifier`
`servicenow.io/application-service-identifier`	**Must**	Application service `identifier`
`servicenow.io/environment`	**Must**	`environment` or `tag_defaults`
`servicenow.io/location`	**Must**	`location` or `tag_defaults`
`servicenow.io/service-tier`	**Should**	`service_tier`
`servicenow.io/cluster`	**Should**	`cluster` (Kubernetes)
`servicenow.io/namespace`	**May**	`namespace` (Kubernetes)

> **Kubernetes — additional keys (`tags.kubernetes`)**

Label key	Requirement
`app.kubernetes.io/name`	**Must**
`app.kubernetes.io/instance`	**Must**
`app.kubernetes.io/component`	**Must**
`app.kubernetes.io/part-of`	**Must**
`app.kubernetes.io/managed-by`	**Should**
`app.kubernetes.io/version`	**May**

> **Docker Compose — additional keys (`tags.docker`)**

Label key	Requirement
`com.docker.compose.project`	**Must**
`com.docker.compose.service`	**Must**

Authors **must** apply the same **servicenow.io/** keys on Docker containers as on Kubernetes pods when using tag-based SM.

1.3 Vertical discovery prerequisites (all platforms)

Vertical discovery **must not** be triggered until all of the following are true:

Prerequisite	Requirement
Entry point CI	**Must** exist (`cmdb_ci_endpoint`) with **Runs on::Runs** to Linux server
sa_m2m registration	**Must** be registered via `SNC.BusinessServiceManager.addEntryPoint`
MID Server	**Must** be up and assigned
Discovery credentials	**Must** be valid for target host
TCP enrichment	**Must** — `cmdb_tcp` (or equivalent) row on entry **port** on target host
Process linkage	**Must** — listener process associated with that TCP row
Docker vertical only	**Must** — Docker Pattern discovery has linked containers to host/process graph (custom container sync alone is **not** sufficient)
Timing	**Should** — horizontal discovery completed **before** vertical trigger (re-trigger vertical if horizontal ran later)

When `service_status=requirements` persists after meeting the above, operators **should** switch to **tag-based** mapping for Docker Compose or investigate shared-tenant SM backlog.

2. CSDM Modeler statements

2.1 Specification files

1. The CSDM Modeler **must** use YAML lists for `business_applications`, `business_services`, and `application_services`.
2. Specifications **may** use Jinja for values resolved from context variable files at deploy time.
3. Application-specific CSDM names, identifiers, and descriptions **must** live in **`csdm.yaml`**, not in shared context registries.
4. Each file **should** live at `<application-playbook-dir>/servicenow/csdm.yaml` and **must** be registered in `csdm/common/vars.yml`.
5. Every BA, BS, and application service **must** declare an **`identifier`** obeying identifier rules.

2.2 Business Application attributes

> **For Business Application (`cmdb_ci_business_app`) attributes:**

Field	Statement
name	**Must** — CMDB display name
identifier	**Must** — stable machine key
business_owner	**Must** — ServiceNow `user_name`; maps to CMDB `owned_by` when no `business_owner` column

Field	Statement
it_application_owner	**Must** — ServiceNow `user_name`
operational_status	**Must** — `"1"` = Operational
active	**Must** — `"true"` / `"false"`
short_description	**Should** — human-readable summary

2.3 Business Service attributes

> **For Business Service (`cmdb_ci_service`) attributes:**

Field	Statement
name	**Must** — CMDB display name
identifier	**Must** — stable machine key
parent_business_application	**Must** — parent BA `name`
owned_by	**Must** — ServiceNow `user_name`
busines_criticality	**Must** — `"1 - most critical"` … `"4 - not critical"`
operational_status	**Must** — `"1"` = Operational
short_description	**Should** — human-readable summary

2.4 Application Service attributes

> **For Application Service (`cmdb_ci_service_discovered`) attributes:**

Field	Statement
name	**Must** — display name; **may** use `{host}` / `{host_lower}` with `expand`
identifier	**Must** — stable machine key; **must** match primary SM tag when tag-based
parent_business_service	**Must** — parent BS `name`
owned_by	**Must** — ServiceNow `user_name`
busines_criticality	**Must** — choice value
operational_status	**Must** — `"1"` = Operational
platform	**Must** — `kubernetes`, `docker`, `host`, or `saas`
environment	**Must** — or inherit from `tag_defaults`
location	**Must** — CMDB location name; **must** match `servicenow.io/location` on workloads
service_mapping	**Must** — `tags`, `vertical`, or `manual` per platform table (Statement 1)

Field	Statement
discover	**Conditional** — **must** be `false` for Kubernetes, SaaS, and tag-based Docker/host; **may** be `true` only with `service_mapping: vertical`
service_tier	**Should** — `web`, `app`, `data`, `ingest`, `control-plane`, `compute`
cluster	**Should** when `platform: kubernetes`
namespace	**May** when `platform: kubernetes`
depends_on	**Should** — tiered consumer → provider list
tags	**Must** when `service_mapping: tags` — nested `kubernetes` and/or `docker` maps
entry_points	**Must** when `service_mapping: vertical` and `discover: true` on Docker/host
expand	**May** — `inventory_group` for per-host instances
short_description	**Should**

2.5 Platform-specific modeling rules

Kubernetes

1. The CSDM Modeler **must** set `platform: kubernetes`, `service_mapping: tags`, and `discover: false`.
2. The CSDM Modeler **must** supply `tags.kubernetes` per Statement 1.2.
3. The CSDM Modeler **must not** declare `entry_points` for Kubernetes application services.

Docker Compose

1. The CSDM Modeler **must** set `platform: docker`.
2. The CSDM Modeler **should** set `service_mapping: tags` and `discover: false` for Compose stacks (ServiceNow recommended path).
3. The CSDM Modeler **may** set `service_mapping: vertical` and `discover: true` only when Statement 1.3 prerequisites are met and ports are stable.
4. When tag-based, the CSDM Modeler **must** declare `tags.docker` and apply **servicenow.io/** labels on Compose services.
5. When vertical, the CSDM Modeler **must** declare `entry_points` with `host_lookup_name` resolving to a Linux server CI.

Host

1. The CSDM Modeler **must** set `platform: host`.
2. The CSDM Modeler **should** prefer `service_mapping: tags` when agents are identified by labels; **may** use vertical when a stable host:port entry point exists and SSH discovery enriches TCP.
3. When vertical with `discover: true`, the CSDM Modeler **must** declare `entry_points`.

SaaS

1. The CSDM Modeler **must** set `platform: saas`, `service_mapping: manual`, and `discover: false`.

2.6 depends_on, csdm_defaults, and csdm_op

1. The CSDM Modeler **must** use `depends_on` on application services for consumer → provider relationships.
2. Each item **may** be a string (application service `name`) or a mapping with `name` and optional `type` (`linux_server`, `nfs_server`, `business_service`).
3. The CSDM Modeler **should** set `csdm_defaults` at file scope for shared owners and criticality.
4. **`csdm_op`**: `insert` (default, upsert) or `delete`. Delete entries **must** include `name`; processor removes relationships then CI.

2.7 Allowed values

busines_criticality: `1 - most critical`, `2 - somewhat critical`, `3 - less critical`, `4 - not critical`

environment: `on-prem`, `dev`, `stage`, `prod`, `lab`

3. Deploy processor statements

3.1 Deploy order

1. The deploy processor **must** run `csdm_op: delete` in order: application services → business services → business applications (before inserts in the same file).
2. The deploy processor **must** create inserts in order: business applications → business services → application services → entry points (when applicable).
3. The deploy processor **must** run a second pass for all `depends_on` after every specification file in the run has been processed.
4. Deferred targets **must not** fail the play; the processor **must** report them.

3.2 CMDB materialization

1. The deploy processor **must** resolve `user_name` values to `sys_user.sys_id` before create or patch.
2. The deploy processor **must** map YAML `business_owner` to CMDB `owned_by` when the instance has no `business_owner` column.
3. The deploy processor **must** create **Contains::Contained by** for BA → BS → application service hierarchy.
4. The deploy processor **must** create **Depends on::Used by** for `depends_on` entries.

3.3 Service Mapping triggers

1. The deploy processor **must** trigger vertical discovery asynchronously when `service_mapping: vertical` and `discover: true`; it **must not** wait for completion.
2. The deploy processor **must not** trigger vertical discovery for `platform: kubernetes`, `service_mapping: tags`, or `service_mapping: manual`.

3. For vertical entry points, the deploy processor **must** create `cmdb_ci_endpoint` with **Runs on::Runs** to host and register via Service Mapping Operations REST API (`addEntryPoint`).
4. For tag-based services, the deploy processor **must not** create entry points or call `addEntryPoint`.
5. The deploy processor **may** emit a runtime tag manifest under `vars/contexts/csdm_runtime_tags/` for downstream label application.

3.4 Validation

1. The deploy processor **must** reject Kubernetes application services that declare `service_mapping: vertical` or `discover: true`.
2. The deploy processor **must** reject SaaS application services unless `service_mapping: manual` and `discover: false`.
3. The deploy processor **must** reject tag-based application services without a `tags` map.
4. The deploy processor **must** reject vertical application services with `discover: true` and no `entry_points`.

4. Discovery operator statements

1. The discovery operator **must** run horizontal Linux Discovery before expecting vertical host or Docker entry-point enrichment.
2. The discovery operator **must** run KVA for Kubernetes clusters before expecting tag-based K8s maps.
3. The discovery operator **should** run ServiceNow **Docker Pattern** on Docker hosts when any application service on that host uses `service_mapping: vertical`.
4. The discovery operator **may** run custom container inventory sync (`discovery/docker/discover.yml`) for CMDB visibility; that sync **does not** replace Docker Pattern for vertical SM.

5. Service Mapping operator statements

1. The Service Mapping operator **must** configure tag-based rules matching **servicenow.io/application-service-identifier** (or the configured prefix) before tag-based application services can reach operational status.
2. The Service Mapping operator **should** monitor `process_status` and `service_status` on `cmdb_ci_service_discovered` after vertical triggers.
3. When migrating Docker services from vertical to tag-based, the operator **should** remove stale entry-point dependencies and reset `process_status` / `service_status` on affected application services.

Commentary

Why identifiers and tags overlap

CSDM display **`name`** values are human-oriented. Tags and Service Mapping rules require stable machine keys. **`identifier`** is the single source of truth; top-level **`environment`**, **`location`**, and **`service_tier`** duplicate into **servicenow.io/** label keys so SM rules do not parse nested YAML.

Why Docker Compose should use tag-based mapping in this lab

ServiceNow **allows** vertical discovery for Docker but **recommends** tags for Compose stacks that change frequently. Vertical discovery requires a traversable graph from entry point → TCP → process → container. Container inventory sync that only upserts `cmdb_ci_docker_container` rows without **cmdb_rel_ci** relationships leaves vertical discovery stuck in **Requirements** — tag-based mapping with Compose labels is the reliable path when **servicenow.io/** labels are already on services.

Deferred linking

Infrastructure targets and cross-file application services may not exist on first deploy. The second-pass linker resolves registry → CMDB lookup → defer without duplicate relationships.

References

- ServiceNow CSDM documentation (Business Application, Business Service, Application Service)
- ServiceNow Quick Start Guide for Service Mapping
- Kubernetes Recommended Labels
- `meta-standards/tpgs-for-tpgs.md` — TPG structure
- `meta-standards/keywords-for-standards.md` — must, should, may